

Hanu Agri: Demand Prediction System

Project Architecture, Tools & Top 50 Viva Questions

1. Project Properties & Architecture

- **Frontend:** Pure HTML5, CSS3, and Vanilla JavaScript. Ensures lightweight, fast rendering without heavy frameworks.
- **Backend:** Python with Flask framework. Serves RESTful APIs for the frontend to consume.
- **Machine Learning Models:** Built using Scikit-Learn (Random Forest classifiers/regressors) for crop and fertilizer recommendations.
- **Computer Vision:** OpenCV and basic Deep Learning/Image Processing for Crop Disease Detection.
- **External APIs:** Uses open-source APIs (like wttr.in for weather) and OpenStreetMap (Leaflet.js) for finding nearby APMC markets.
- **Multi-language Support:** Built-in i18n system in JavaScript to toggle between English, Hindi, and Kannada.
- **Architecture Pattern:** Client-Server architecture with stateless REST APIs.

2. Tools & Technologies Used

- **Programming Languages:** Python 3.12, JavaScript (ES6+)
- **Libraries (Python):** Flask, Flask-CORS, Pandas, NumPy, Scikit-Learn, OpenCV (opencv-python-headless), urllib
- **Libraries (JS):** Leaflet.js (for interactive maps), Chart.js (for data visualization)
- **Version Control & Deployment:** Git, GitHub, Render (PaaS), Railway
- **Data Processing:** Pandas for data cleaning and mock dataset generation
- **Development Environment:** VS Code, Chrome Developer Tools

3. Top 50 Viva / Interview Questions

Q1: What is the main objective of the Hanu Agri project?

Ans: To empower farmers with AI-driven insights for crop selection, fertilizer use, and market prices to maximize profitability.

Q2: Why did you choose Flask over Django for the backend?

Ans: Flask is a lightweight micro-framework, which is perfect for serving simple REST APIs for our Machine Learning models without unnecessary overhead.

Q3: What real-world problem does this project solve?

Ans: It solves the lack of data-driven decision making in traditional farming, helping farmers avoid crop failure and market gluts.

Q4: How does the system ensure it is accessible to rural farmers?

Ans: By providing multi-language support (English, Hindi, Kannada) and an easy-to-use intuitive UI.

Q5: What architectural pattern is used in this project?

Ans: Client-Server architecture. The frontend is fully decoupled from the backend and communicates via REST APIs.

Q6: Why did you use Vanilla JavaScript instead of React or Angular?

Ans: To keep the project lightweight, minimize dependencies, and ensure fast load times even on slow networks.

Q7: How is multi-language support implemented?

Ans: A JavaScript dictionary stores translations. A function traverses the DOM elements with a 'data-i18n' attribute and updates their inner text based on the selected language.

Q8: How does the frontend communicate with the backend?

Ans: Using the JavaScript Fetch API to send GET and POST requests to the Flask server endpoints.

Q9: What is Leaflet.js used for in your project?

Ans: It is used in the 'Market Finder' module to render an interactive map and display the user's location alongside nearby APMC markets.

Q10: How is the user's location retrieved?

Ans: Using the HTML5 Geolocation API (`navigator.geolocation.getCurrentPosition`).

Q11: Which machine learning algorithm is used for Crop Recommendation?

Ans: Random Forest Classifier. It handles non-linear data well and is robust to overfitting.

Q12: What features are used to predict the best crop?

Ans: Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall.

Q13: How did you handle the lack of real-world agricultural data?

Ans: We wrote a Python script to synthesize highly realistic datasets based on known agricultural parameters for Indian crops.

Q14: Why is Random Forest preferred over Decision Trees here?

Ans: Random Forest builds multiple decision trees and averages them, preventing the overfitting that a single decision tree is prone to.

Q15: How does the Fertilizer Guidance module work?

Ans: It takes the current soil nutrients and crop type, compares them to ideal thresholds, and recommends specific fertilizers (like Urea or DAP) to bridge the gap.

Q16: What preprocessing is applied to the data before training?

Ans: Data is scaled using StandardScaler to normalize feature ranges, and labels are encoded.

Q17: How is the trained model integrated into the Flask app?

Ans: The model and scaler are serialized using 'pickle' and loaded into memory when the Flask server starts.

Q18: What happens if a user enters extremely out-of-bound soil values?

Ans: The Random Forest model will predict the closest matching crop, but the system also has threshold checks to warn users if the land is uncultivable.

Q19: How does the Price Prediction module work?

Ans: It uses historical mock data and a regression model to forecast the price of a commodity for upcoming months.

Q20: What is the Demand & Glut Risk module?

Ans: It analyzes current market trends and advises farmers if planting a specific crop might lead to an oversupply (glut) and low prices.

Q21: How do you calculate ROI (Return on Investment)?

Ans: By subtracting estimated cultivation costs from the predicted market revenue ($\text{Price} \times \text{Expected Yield}$).

Q22: How does the Crop Disease Detection module work?

Ans: The user uploads an image. OpenCV is used to process the image, extract color/texture features, and classify the disease.

Q23: Why use OpenCV instead of a CNN (like ResNet)?

Ans: OpenCV allows for lightweight, rule-based color masking (e.g., detecting yellowing or brown spots) which runs very fast without needing a GPU.

Q24: What image preprocessing steps are performed?

Ans: Resizing, Gaussian blurring to reduce noise, and converting from BGR to HSV color space for better color segmentation.

Q25: How does the AI Chatbot understand user queries?

Ans: It uses Regex and keyword matching to identify intents like 'weather' or 'fertilizer prices'.

Q26: Which API is used for weather fetching?

Ans: The wttr.in API, which provides free weather data without requiring an API key.

Q27: Why did you switch from Open-Meteo/Nominatim to wttr.in?

Ans: Cloud hosting providers (like Render) often get IP-blocked by Nominatim. wttr.in is more cloud-friendly for fetching weather.

Q28: How does the chatbot handle unsupported questions?

Ans: It has a fallback default response guiding the user to ask about supported topics like weather or crop prices.

Q29: What is the role of Gunicorn in this project?

Ans: Gunicorn is a Python Web Server Gateway Interface (WSGI) HTTP server. It is used to serve the Flask app in production, handling multiple concurrent requests.

Q30: Why can't we just use 'app.run()' in production?

Ans: Flask's built-in server is not designed for production; it is single-threaded and cannot handle concurrent traffic efficiently.

Q31: How is the project deployed?

Ans: It is deployed as a Web Service on Render, which automatically builds the environment from 'requirements.txt' and runs Gunicorn.

Q32: What is the purpose of the 'render.yaml' file?

Ans: It is an Infrastructure-as-Code Blueprint that tells Render exactly how to build and start the application.

Q33: Why are the ML models trained during the 'build' phase on Render?

Ans: Because Gunicorn doesn't run the 'if __name__ == __main__' block. Training during the build ensures models exist before workers start, preventing file-not-found errors.

Q34: What is CORS and why is it used?

Ans: Cross-Origin Resource Sharing. It's enabled in Flask to allow the frontend (if hosted on a different domain) to make API requests to the backend.

Q35: What happens if the weather API goes down?

Ans: The backend wraps the request in a try-except block and returns a graceful error message instead of crashing the server.

Q36: How do you handle missing environment variables?

Ans: We provide default fallback values in the code, though Render injects necessary variables like PORT automatically.

Q37: What is 'pickle' in Python?

Ans: A module used for serializing and de-serializing Python object structures, used here to save trained ML models.

Q38: Why use 'np.argsort()[::-1]' in predictions?

Ans: It sorts the prediction probabilities in descending order so we can recommend the top 3 or top 5 best crops.

Q39: How does the 'Harvest Readiness' module analyze videos?

Ans: It extracts frames using OpenCV (`cv2.VideoCapture``), analyzes the color distribution of the crop in those frames, and averages the results to determine maturity.

Q40: How is state managed in the frontend?

Ans: State (like current language and current page) is managed using global variables in 'app.js'.

Q41: What is the accuracy of your Crop Recommendation model?

Ans: Typically around 95-98% on our synthesized dataset.

Q42: How do you evaluate a classification model?

Ans: Using metrics like Accuracy, Precision, Recall, and F1-Score, derived from a Confusion Matrix.

Q43: How would you improve the Disease Detection module?

Ans: By replacing OpenCV color masking with a deep learning CNN model like MobileNet trained on the PlantVillage dataset.

Q44: What is the biggest limitation of your current system?

Ans: It relies heavily on synthesized data. Real-world implementation requires integration with live IoT soil sensors and real APMC market data APIs.

Q45: How could you make the chatbot smarter?

Ans: By integrating a Large Language Model (LLM) API like Google Gemini or OpenAI to handle conversational context naturally.

Q46: How is user data security handled?

Ans: Currently, the app doesn't require user logins or save personal data, ensuring complete privacy.

Q47: What is the role of CSS Variables in your project?

Ans: CSS variables (e.g., `--color-primary`) define the theme. They make it extremely easy to maintain consistency and implement Dark Mode in the future.

Q48: How is the app made responsive for mobile devices?

Ans: Using CSS media queries, flexbox, and a toggleable mobile sidebar.

Q49: Why do you use 'object-fit: cover' for images?

Ans: It ensures images (like the background or profile photo) fill their container without distorting their aspect ratio.

Q50: What was the most challenging bug you faced during deployment?

Ans: The models failing to load on Render because Unicorn bypasses the main execution block, which was solved by adding a dedicated setup script during the build phase.